Exercise 12 — Function Decomposition Challenge

Exercise Description

Select a complex function, use AI to identify its distinct responsibilities, create a decomposition plan, extract well-named helper functions, refactor the main function to call them, **run tests to verify behaviour is preserved**, and document the approach and benefits.

Function chosen: `generate_sales_report` (Python, `refactor-functions/python/sales_report.py`) — a single ~200-line function.

Files: - `sales_report_original.py` — the untouched original. - `sales_report.py` — the decomposed version. - `test_sales_report.py` — the original test suite (unchanged), used as the behaviour-preservation harness.

Distinct responsibilities identified

The original mixed **nine** responsibilities in one function: 1. input validation, 2. date-range filtering, 3. additional filtering, 4. empty-data handling, 5. basic metrics, 6. grouping, 7. report-type sections (summary/detailed/forecast), 8. charts, 9. rendering by output format.

Decomposition plan → extracted helpers

Helper	Responsibility
`_validate_inputs`	Validate `sales_data`, `report_type`, `output_format`
`_apply_date_range`	Filter to `[start, end]` (and validate the range)
`_apply_filters`	Apply equality / list-membership filters
`_basic_metrics` / `_sale_summary`	Total, average, max/min sale
`_group_sales`	Group by field with count/total/items/average
`_grouping_section`	Build the grouping block (with percentages)
`_transaction_details`	Detailed-report per-transaction calculated fields
`_forecast_section`	Monthly aggregation, growth rates, 3-month projection
`_charts_section`	Sales-over-time + optional grouping pie
`_render`	Dispatch to the output-format renderer

`generate_sales_report` is now a thin **orchestrator** that validates, filters, computes metrics, and conditionally attaches sections.

Verification — behaviour preserved

The original test suite passes unchanged against the refactored code:

```
Ran 8 tests in 0.013s
OK
```

(The "No data matches" warning printed after the run comes from the empty-filter test, exactly as before.)

Benefits gained

- **Testability:** each section (metrics, grouping, forecast, charts) can now be unit-tested in isolation.
- **Readability:** the orchestrator reads as a table of contents; the ~200-line wall is gone.
- **Change isolation:** e.g. tweaking the forecast horizon touches only `_forecast_section`.
- **Reuse:** `_apply_filters` / `_basic_metrics` are reusable by other report entry points.

Reflection Questions

1. **How did decomposition improve readability/maintainability?** The main function now expresses *what* a report is (validate → filter → metrics → sections), while each *how* lives in a named, testable helper.
2. **Most challenging part?** The forecast block — it carries mutable running state (`last_amount`, rolling month/year). It had to be extracted as a whole to preserve the exact projection math and output ordering.
3. **Which extracted function is most reusable?** `_apply_filters` and `_basic_metrics` — generic over any sales-like list, useful well beyond this one report.

Submission for Exercise 12 — Function Decomposition Challenge

1. Distinct responsibilities identified: the nine listed above (validation, two filtering stages, empty-data handling, metrics, grouping, per-type sections, charts, rendering).

2. Decomposition plan: the helper table above, each with a single responsibility.

3. Extracted helper functions: implemented in `sales_report.py` (`_validate_inputs`, `_apply_date_range`, `_apply_filters`, `_basic_metrics`, `_group_sales`, `_grouping_section`, `_transaction_details`, `_forecast_section`, `_charts_section`, `_render`).

4. Refactored main function + tests: `generate_sales_report` reduced to an orchestrator; the original suite passes unchanged — **8 tests, OK** — confirming behaviour is preserved.

5. Approach & benefits: extract-and-verify against the existing tests; benefits are isolated testability, a readable orchestrator, change isolation, and reusable filter/metric helpers (reflection answers above).